

Lecture 4: GitHub Actions CD for ML Inference Service

Champion Model → Docker Image → Download → Run Locally
→ Inference API

MLOps Foundations

1. Lecture Goal

In Lecture 1, we used GitHub Actions to:

1. train a model,
2. evaluate it,
3. save `model.pkl`,
4. simulate a registry using `champion.json`,
5. upload artifacts.

In this lecture, we extend that pipeline into a simple **continuous delivery** workflow for ML systems.

After the champion model is produced, we will:

1. build a FastAPI inference service around the champion model,
2. containerize it with Docker,
3. save the Docker image as a `.tar` artifact in GitHub Actions,
4. download the image locally,
5. load it into Docker Desktop,
6. run the container,
7. call the inference API and get predictions.

2. What This Lecture Teaches

This lecture shows students that deployment in MLOps means more than saving `model.pkl`. A trained model is useful only when it is packaged together with:

- the inference code,
- the dependencies,

- the runtime environment,
- the API interface.

The final deliverable is not just a file. It is a runnable **Docker image**.

3. Overall Pipeline

The complete pipeline in this lecture is:

Git push → train model → evaluate model → save `model.pkl` → update `champion.json` →
prepare deployment files → build Docker image → save image as `tar` → download artifact →
`docker load` → `docker run` → call inference API

4. Project Structure

We will extend the project from Lecture 1 into this structure:

```
mlops-github-actions-demo/  
  
  .github/  
    workflows/  
      ml_cd_pipeline.yml  
  
  src/  
    train.py  
  
  api/  
    app.py  
  
  outputs/  
    model.pkl  
    metrics.json  
  
  registry/  
    champion.json  
  
  deployment/  
    model.pkl  
    app.py  
    requirements-api.txt  
    Dockerfile  
  
  requirements.txt  
  README.md
```

5. Step 1: Start from the Lecture 1 Project

Make sure you already have the Lecture 1 project with:

- `src/train.py`

- outputs/model.pkl
- outputs/metrics.json
- registry/champion.json

If needed, run the training script locally first:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
python src/train.py
```

6. Step 2: Create New Folders

Create the folders for the inference API and deployment packaging:

```
mkdir -p api
mkdir -p deployment
mkdir -p .github/workflows
```

7. Step 3: Create the Inference API

Create api/app.py.

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
import os

MODEL_PATH = os.path.join(os.path.dirname(__file__), "model.pkl")
model = joblib.load(MODEL_PATH)

app = FastAPI(title="Champion Model Inference API")

class PredictionRequest(BaseModel):
    features: list[float]

@app.get("/")
def root():
    return {"message": "Champion model inference API is running."}

@app.post("/predict")
def predict(request: PredictionRequest):
    prediction = model.predict([request.features])[0]
    return {"prediction": int(prediction)}
```

8. Step 4: Create API Requirements

Create deployment/requirements-api.txt.

```
fastapi
uvicorn[standard]
scikit-learn
joblib
pydantic
```

9. Step 5: Create the Dockerfile

Create deployment/Dockerfile.

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements-api.txt .
RUN pip install --no-cache-dir -r requirements-api.txt

COPY app.py .
COPY model.pkl .

EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

10. Step 6: Understand the Dockerfile

This Dockerfile does the following:

1. starts from a lightweight Python base image,
2. sets the working directory inside the container,
3. copies the API dependency file,
4. installs FastAPI, Uvicorn, scikit-learn, and joblib,
5. copies the inference code,
6. copies the champion model file,
7. exposes port 8000,
8. starts the FastAPI app with Uvicorn.

11. Step 7: Prepare the Deployment Files

We now need a script that copies the champion model into the deployment folder and prepares everything needed for Docker build.

Create `src/prepare_deployment.py`.

```
import json
import shutil
from pathlib import Path

registry_path = Path("registry/champion.json")
deployment_dir = Path("deployment")
api_dir = Path("api")

deployment_dir.mkdir(parents=True, exist_ok=True)

if not registry_path.exists():
    raise FileNotFoundError("registry/champion.json not found")

with open(registry_path, "r") as f:
    champion = json.load(f)

model_source = Path(champion["model_path"])
if not model_source.exists():
    raise FileNotFoundError(f"Champion model not found at {model_source}")

# Copy champion model into deployment folder
shutil.copy2(model_source, deployment_dir / "model.pkl")

# Copy API app into deployment folder
shutil.copy2(api_dir / "app.py", deployment_dir / "app.py")

print("Deployment files prepared successfully.")
print(f"Champion model copied from: {model_source}")
print("Files ready inside deployment/")
```

12. Step 8: Test the API Locally Without Docker

Before building the container, verify the API locally.

First copy the champion model manually for testing:

```
cp outputs/model.pkl api/model.pkl
```

Then install the API dependencies:

```
pip install fastapi "uvicorn[standard]" scikit-learn joblib pydantic
```

Run the API:

```
uvicorn api.app:app --host 0.0.0.0 --port 8000
```

Open another terminal and test the root endpoint:

```
curl http://127.0.0.1:8000/
```

Test prediction:

```
curl -X POST "http://127.0.0.1:8000/predict" \
-H "Content-Type: application/json" \
-d '{
```

```
"features": [  
  17.99, 10.38, 122.8, 1001.0,  
  0.1184, 0.2776, 0.3001, 0.1471,  
  0.2419, 0.0787, 1.095, 0.9053,  
  8.589, 153.4, 0.006399, 0.04904,  
  0.05373, 0.01587, 0.03003, 0.006193,  
  25.38, 17.33, 184.6, 2019.0,  
  0.1622, 0.6656, 0.7119, 0.2654,  
  0.4601, 0.1189  
]  
'}
```

13. Step 9: Build the Docker Image Locally First

First copy the champion model and app.py manually to the deployment folder:

```
cp api/app.py deployment/app.py  
cp outputs/model.pkl deployment/model.pkl
```

Move into the deployment folder:

```
cd deployment
```

Build the Docker image:

```
docker build -t champion-inference-api:latest .
```

Run the container:

```
docker run -p 8000:8000 champion-inference-api:latest
```

Open another terminal and test it:

```
curl http://127.0.0.1:8000/
```

```
curl -X POST "http://127.0.0.1:8000/predict" \  
-H "Content-Type: application/json" \  
-d '{  
  "features": [  
    17.99, 10.38, 122.8, 1001.0,  
    0.1184, 0.2776, 0.3001, 0.1471,  
    0.2419, 0.0787, 1.095, 0.9053,  
    8.589, 153.4, 0.006399, 0.04904,  
    0.05373, 0.01587, 0.03003, 0.006193,  
    25.38, 17.33, 184.6, 2019.0,  
    0.1622, 0.6656, 0.7119, 0.2654,  
    0.4601, 0.1189  
  ]  
'}
```

Stop the container when finished.

14. Step 10: Create the GitHub Actions Workflow for CD

Now create `.github/workflows/ml_cd_pipeline.yml`.

```
name: ML CD Pipeline for Champion Model API

on:
  push:
    branches:
      - main
  workflow_dispatch:

jobs:
  train-and-register:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install training dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run training pipeline
        run: |
          python src/train.py

      - name: Upload training artifacts
        uses: actions/upload-artifact@v4
        with:
          name: training-outputs
          path: |
            outputs/model.pkl
            outputs/metrics.json
            registry/champion.json

  build-inference-image:
    runs-on: ubuntu-latest
    needs: train-and-register

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
```

```
python-version: "3.11"

- name: Download training artifacts
  uses: actions/download-artifact@v4
  with:
    name: training-outputs
    path: .

- name: Prepare deployment files
  run: |
    python src/prepare_deployment.py

- name: Show deployment contents
  run: |
    ls -R deployment

- name: Build Docker image
  run: |
    docker build -t champion-inference-api:latest deployment

- name: Save Docker image as tar
  run: |
    docker save -o champion-inference-api.tar champion-inference-api:latest

- name: Upload Docker image artifact
  uses: actions/upload-artifact@v4
  with:
    name: champion-docker-image
    path: champion-inference-api.tar
```

15. Step 11: Understand the Workflow

This workflow has two jobs.

15.1. Job 1: train-and-register

This job:

1. checks out the repository,
2. installs Python,
3. installs training dependencies,
4. runs `src/train.py`,
5. uploads the model, metrics, and champion registry file as artifacts.

15.2. Job 2: build-inference-image

This job runs only after Job 1 succeeds.

It:

1. downloads the artifacts from Job 1,
2. prepares deployment files,
3. builds a Docker image containing the API and champion model,
4. saves the image into a `.tar` file,
5. uploads that tar file as a GitHub Actions artifact.

16. Step 12: Commit and Push

Now commit everything:

```
git add .
git commit -m "Add GitHub Actions CD pipeline for champion model API"
git push origin main
```

After pushing, go to the GitHub repository and open the **Actions** tab.
You should see the workflow run:

- first the training job,
- then the Docker build job.

17. Step 13: Download the Docker Image Artifact

After the workflow finishes:

1. open the workflow run in GitHub,
2. find the artifact named `champion-docker-image`,
3. download it,
4. extract it if GitHub gives it as a zip. If you are not getting a `.tar` file that means your extractor is extracting both the zip and the tar file together.. Use something like <https://zipextractor.app/> to extract the zip file and get the `.tar` file
5. locate the file `champion-inference-api.tar`.

18. Step 14: Load the Docker Image into Docker Desktop

From your terminal, navigate to the folder containing the tar file and run:

```
docker load -i champion-inference-api.tar
```

Check that the image is now available:

```
docker images
```

You should see something like:

```
champion-inference-api    latest    ...
```

19. Step 15: Run the Docker Container Locally

Run the container on port 8000:

```
docker run -p 8000:8000 champion-inference-api:latest
```

Now the API should be accessible locally.

20. Step 16: Test the Inference API

Test the root endpoint:

```
curl http://127.0.0.1:8000/
```

Expected response:

```
{"message": "Champion model inference API is running."}
```

Now test the prediction endpoint:

```
curl -X POST "http://127.0.0.1:8000/predict" \
-H "Content-Type: application/json" \
-d '{
  "features": [
    17.99, 10.38, 122.8, 1001.0,
    0.1184, 0.2776, 0.3001, 0.1471,
    0.2419, 0.0787, 1.095, 0.9053,
    8.589, 153.4, 0.006399, 0.04904,
    0.05373, 0.01587, 0.03003, 0.006193,
    25.38, 17.33, 184.6, 2019.0,
    0.1622, 0.6656, 0.7119, 0.2654,
    0.4601, 0.1189
  ]
}'
```

Example response:

```
{"prediction": 0}
```

21. In Class Exercise

Complete everything above and show me the downloaded docker container with inference working on your PC.